

CHAPTER - 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Heads Up For Tails (HUFT) E-Commerce Order Management Database System is designed to efficiently manage customers, products, orders, payments, shipments, and administrators. Functional dependencies help identify the relationship between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and reduce duplication. A well-designed database improves performance, maintains data integrity, and supports efficient order management.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- Insert Anomaly: A new customer can be added without creating an order.
- Update Anomaly: Updating a customer's phone number requires changes only in the Customer table.
- Delete Anomaly: Deleting an order will not remove customer information.

2. CATEGORY TABLE

Functional Dependency

Category_ID $\rightarrow$ Category_Name

Redundancy Analysis

Each category is stored only once. Products reference the category using Category_ID instead of repeating the category name.

Data Anomalies

- Insert Anomaly: New categories can be added without products.
- Update Anomaly: Renaming a category is done only once.
- Delete Anomaly: Deleting a product does not delete the category.

3. PRODUCT TABLE

Functional Dependency

Product_ID $\rightarrow$ Product_Name, Price, Stock, Category_ID

Redundancy Analysis

Product details are stored only in the Product table. Order details use Product_ID instead of repeating product information.

Data Anomalies

- Insert Anomaly: New products can be added before receiving orders.
- Update Anomaly: Product price is updated only once.
- Delete Anomaly: Deleting an order does not remove product information.

4. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order-related information. Customer and product details are referenced using foreign keys.

Data Anomalies

- Insert Anomaly: Orders can be created independently using Customer_ID.
- Update Anomaly: Order status can be updated without affecting other tables.
- Delete Anomaly: Deleting an order does not remove customer or product data.

5. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID → Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

This table connects orders and products, avoiding repeated product information in the Orders table.

Data Anomalies

- Insert Anomaly: Products can be added to an order independently.
- Update Anomaly: Quantity changes affect only the selected order item.
- Delete Anomaly: Removing an order item does not delete the product.

6. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status

Redundancy Analysis

Payment information is stored separately from the Orders table to avoid duplicate payment records.

Data Anomalies

- Insert Anomaly: Payment details can be recorded after creating an order.
- Update Anomaly: Payment status is updated only once.
- Delete Anomaly: Deleting payment information does not remove the order.

7. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID, Tracking_Number, Delivery_Status

Redundancy Analysis

Shipment details are maintained in a separate table, preventing repeated delivery information.

Data Anomalies

- Insert Anomaly: Shipment records can be added after order confirmation.
- Update Anomaly: Delivery status can be updated independently.
- Delete Anomaly: Deleting shipment information does not affect order details.

8. ADMIN TABLE

Functional Dependency

Admin_ID → Admin_Name, Email, Password

Redundancy Analysis

Administrator information is stored separately to avoid repeating login details across different tables.

Data Anomalies

- Insert Anomaly: New administrators can be added independently.
- Update Anomaly: Admin details are updated only once.
- Delete Anomaly: Removing an administrator does not affect customer or order records.

CONCLUSION

The HUFT E-Commerce Order Management Database System is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant data, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains integrity, and ensures efficient database management for the e-commerce system.